Unit-2 (2 marks)

1. **What is collection Framework. List any two goals of collection Framework.**

The **Collections Framework** is a sophisticated hierarchy of interfaces and classes that provide state-of-the-art technology for managing groups of objects.

Goals:

- First, **the framework had to be high-performance**. The implementations for the fundamental collections (dynamic arrays, linked lists, trees, and hash tables) are highly efficient.
- Second, the framework had to allow different types of collections to work in a similar manner and with a **high degree of interoperability.**
- Third, **extending and/or adapting a collection had to be easy.** Several standard implementations (such as **LinkedList, HashSet, and TreeSet)** of these interfaces are provided that you may use as-is.

2. **What are the benefits of using the Collections Framework?**

It merits close attention by all programmers. Because **java.util** contains a wide array of functionality, it is quite large. **java.util**.*; package contains a large time, observe events, manipulate sets of bits, tokenize strings, and handle formatted data.

3. **List any two collection classes with its purpose.**

Class	Description
AbstractCollection	Implements most of the Collection interface.
LinkedList	Implements a linked list by extending AbstractSequentialList.
ArrayList	Implements a dynamic array by extending AbstractList.
EnumSet	Extends AbstractSet for use with enum elements.

4. **Write the differences between hasNext() and next() method.**

boolean hasNext( ) Returns true if there is a next element. Otherwise, returns false

E next() Returns the next element. A **NoSuchElementException** is thrown if there is not a next element.

5. What is map? List any two Map interface

A **map** is an object that stores associations between keys and values, or key/value pairs. Given a key, you can find its value. Both keys and values are objects. The keys must be unique, but the values may be duplicated.

Interface	Description
Map	Maps unique keys to values.
Map.Entry	Describes an element (a key/value pair) in a map. This is an inner class of Map.
NavigableMap	Extends SortedMap to handle the retrieval of entries based on closest-match searches. (Added by Java SE 6.)
SortedMap	Extends Map so that the keys are maintained in ascending order.

6. Write the purpose of Array class.

The **Arrays** class provides various methods that are useful when working with arrays. These methods help bridge the gap between collections and arrays

7. Write the purpose of

a. **fill()**

b. **copyOf()**

a) The **fill( )** method assigns a value to all elements in an array. In other words, it fills an array with a specified value. The fill( ) method has two versions. The first version, which has the following forms, fills an entire array:

static void fill(boolean *array*[ ], boolean *value*)
static void fill(byte *array*[ ], byte *value*)
static void fill(char *array*[ ], char *value*)
static void fill(double *array*[ ], double *value*)
static void fill(float *array*[ ], float *value*)
static void fill(int *array*[ ], int *value*)
static void fill(long *array*[ ], long *value*)

static void fill(short *array*[ ], short *value*)
static void fill(Object *array*[ ], Object *value*)
Here, *value* is assigned to all elements in *array*.

b) The **copyOf( )** method was added by Java SE 6. It returns a copy of an array and has the following forms:
static boolean[ ] copyOf(boolean[ ] *source*, int *len*)
static byte[ ] copyOf(byte[ ] *source*, int *len*)
static char[ ] copyOf(char[ ] *source*, int *len*)
static double[ ] copyOf(double[ ] *source*, int *len*)
static float[ ] copyOf(float[ ] *source*, int *len*)
static int[ ] copyOf(int[ ] *source*, int *len*)
static long[ ] copyOf(long[ ] *source*, int *len*)
static short[ ] copyOf(short[ ] *source*, int *len*)

8. Write the usage of iterator interface

- Each of the collection classes provides an **iterator( )** method that returns an iterator to the start of the collection.
- By using this iterator object, you can access each element in the collection, one element at a time.
- Iterator enables you to cycle through a collection, obtaining or removing elements.
- ListIterator extends Iterator to allow bidirectional traversal of a list, and the modification of elements.
- Iterator and ListIterator are generic interfaces which are declared as shown here:
- **interface Iterator<E> interface ListIterator<E>**

9. List any four interfaces provided by Collection Framework

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends Queue to handle a double-ended queue. (Added by Java SE 6.)
List	Extends Collection to handle sequences (lists of objects).

NavigableSet	Extends SortedSet to handle retrieval of elements based on closest-match searches. (Added by Java SE 6.)
Queue	Extends Collection to handle special types of lists in which elements are removed only from the head.
Set	Extends Collection to handle sets, which must contain unique elements.
SortedSet	Extends Set to handle sorted sets.

10. Write any two uses of Generics.

- Generics add the one feature that collections had been missing: **type safety**. Prior to generics, all collections stored Object references, which meant that any collection could store any type of object. Thus, it was possible to accidentally store incompatible types in a collection. Doing so could result in run-time type mismatch errors. With generics, it is possible to explicitly state the type of data being stored, and run-time type mismatch errors can be avoided.
- Although the addition of generics changed the declarations of most of its **classes** and **interfaces,** and several of their **methods,** overall, the Collections Framework still works the same as it did prior to generics.

11. List any four exceptions thrown in the context of collections

- If an attempt is made to use one of these methods on an unmodifiable collection, an **UnsupportedOperationException** is thrown.
- A **ClassCastException** is generated when one object is incompatible with another, such as when an attempt is made to add an incompatible object to a collection
- A **NullPointerException** is thrown if an attempt is made to store a null object and null elements are not allowed in the collection.
- An **IllegalArgumentException** is thrown if an invalid argument is used.

- An **IllegalStateException** is thrown if an attempt is made to add an element to a fixed-length collection that is full.

12. What is the usage of containsAll() and retainAll() methods?

boolean containsAll(Collection <?> c) Returns true if the invoking collection contains all elements of c. Otherwise, returns false.

boolean retainAll(Collection<?> c) Removes all elements from the invoking collection except those in c. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.

13. List any four methods of List interface

Method
void add(int index, E obj)
boolean addAll(int index, Collection<? extends E> c)
E get(int index)
int indexOf(Object obj)
int lastIndexOf(Object obj)
ListIterator<E> listIterator( )
ListIterator<E> listIterator(int index)
E remove(int index)
E set(int index, E obj)
List<E> subList(int start, int end)

14. What is the purpose of the NavigableSet interface in the Java Collections Framework.

- **NavigableSet** interface extends **SortedSet** and declares the behavior of a collection that supports the retrieval of elements based on the closest match to a given value or values. **NavigableSet** is a generic interface that has this declaration: **interface NavigableSet<E>**
  Here, E specifies the type of objects that the set will hold

15. How you can add or remove element to/from the first, last using LinkedList Class.

- To add elements to the start of a list you can use **addFirst( )** or **offerFirst( ).**
- To add elements to the end of the list, use **addLast( )** or **offerLast( ).**
- To remove the first element, use **removeFirst( )** or **pollFirst( ).**
- To remove the last element, use **removeLast( )** or **pollLast( ).**

16. Differentiate headset() and tailset() methods

If you need the subset that starts with the first element in the set, use **headSet( ).**

SortedSet<E> headSet(E end) Returns a SortedSet containing those elements less than end that are contained in the invoking sorted set. Elements in the returned sorted set are also referenced by the invoking sorted set.

If you want the subset that ends the set, use **tailSet( ).**

SortedSet<E> tailSet(E start) Returns a SortedSet that contains those elements greater than or equal to start that are contained in the sorted set. Elements in the returned set are also referenced by the invoking object.

17. Differentiate poll() and remove() methods of Queue interface

There are two methods that obtain and remove elements:

poll( ) and remove( ).

The difference between them is that poll( ) returns null if the queue is empty, but remove( ) throws an exception.

E poll( )	Returns the element at the head of the queue, removing the element in the process. It returns null if the queue is empty.
E remove( )	Removes the element at the head of the queue, returning the element in the process. It throws **NoSuchElementException** if the queue is empty.

18. List any four methods of deque interface

Method	Description
void addFirst(E obj)	Adds obj to the head of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
void addLast(E obj)	Adds obj to the tail of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
Iterator<E> descendingIterator( )	Returns an iterator that moves from the tail to the head of the deque. In other words, it returns a reverse iterator.
E getFirst( )	Returns the first element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
E getLast( )	Returns the last element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
boolean offerFirst(E obj)	Attempts to add obj to the head of the deque. Returns true if obj was added and false otherwise. Therefore, this method returns false when an attempt is made to add obj to a full, capacity-restricted deque.
boolean offerLast(E obj)	Attempts to add obj to the tail of the deque. Returns true if obj was added and false otherwise.
E peekFirst( )	Returns the element at the head of the deque. It returns null if the deque is empty. The object is not removed.

19. What is the purpose of push() and pop() methods of Deque interface?

E pop( )	Returns the element at the head of the deque, removing it in the process. It throws **NoSuchElementException** if the deque is empty.
void push(E obj)	Adds obj to the head of the deque. Throws an **IllegalStateException** if a capacity-restricted deque is out of space.

20. How does an ArrayList differ from standard arrays?

- Java ArrayList class uses a *dynamic array* for storing the elements.
- It is like an array, but there is *no size limit*. We can add or remove elements anytime. So, it is much more flexible than the traditional array.
- The ArrayList in Java can have the duplicate elements also.
- It implements the List interface so we can use all the methods of List interface here.
- **ArrayList<String> list=new ArrayList<String>();**

21. What are the syntax/signatures of the two overloaded toArray() methods in the ArrayList class?

There are two versions of toArray( ) methods in the ArrayList class

Object[ ] toArray( )

<T>T[ ] toArray(T array[ ])

The first returns an array of Object. The second returns an array of elements that have the same type as T

22. What is the difference between the HashSet() and HashSet(int capacity) constructors?

HashSet( ) constructs a default hash set

HashSet(int capacity) initializes the capacity of the hash set to capacity. (The default capacity is 16.)

23. What is the purpose of the float fillRatio parameter in the HashSet(int capacity, float fillRatio) constructor?

- **HashSet(int capacity, float fillRatio)** initializes both the capacity and the fill ratio (also called load capacity) of the hash set from its arguments.

- The fill ratio must be between 0.0 and 1.0, and it determines how full the hash set can be before it is resized upward.

24. What is the primary advantage of using a TreeSet over other Set implementations like HashSet?

- TreeSet extends **AbstractSet** and implements the **NavigableSet** interface.
- It creates a collection that uses a tree for storage.
- Objects are stored in sorted, ascending order.
- Access and retrieval times are quite fast, which makes **TreeSet** an excellent choice when storing large amounts of sorted information that must be found quickly.
- TreeSet is a generic class that has this declaration: class **TreeSet<E> h**ere, E specifies the type of objects that the set will hold

25. What is the main difference between using a foreach loop and an Iterator to traverse the elements of a Collection in Java?

- If you won't be modifying the contents of a collection or obtaining elements in reverse order, then the for-each version of the for loop is often a more convenient alternative to cycling through a collection than is using an *iterator.*
- Because all of the collection classes implement this interface, they can all be operated upon by the for.

26. What is the purpose of the RandomAccess interface in Java collections?

- The RandomAccess interface contains no members.
- By implementing this interface, a collection signals that it supports efficient random access to its elements.
- By checking for the RandomAccess interface, client code can determine at run time whether a collection is suitable for certain types of random access operations—especially as they apply to large collections
- RandomAccess is implemented by ArrayList and by the legacy Vector class, among others

27. How does a Map differ from a Collection in Java?

- A map is an object that stores associations between keys and values, or key/value pairs. Given a key, you can find its value.
- Both keys and values are objects.
- The keys must be unique, but the values may be duplicated.
- Some maps can accept a null key and null values, others cannot.
- Keys don't implement the *Iterable* interface. This means that you cannot cycle through a map using a for-each style for loop.

28. List any four Map classes

Interface	Description
Map	Maps unique keys to values.
Map.Entry	Describes an element (a key/value pair) in a map. This is an inner class of Map.
NavigableMap	Extends SortedMap to handle the retrieval of entries based on closest-match searches. (Added by Java SE 6.)
SortedMap	Extends Map so that the keys are maintained in ascending order.

29. What is the purpose of using a Comparator with TreeSet and TreeMap in Java?

- Both TreeSet and TreeMap store elements in sorted order.
- If you want to order elements a different way, then specify a **Comparator** when you construct the set or map.
- Doing so gives you the ability to govern precisely how elements are stored within sorted collections and maps.
- Comparator is a generic interface that has this declaration: **interface Comparator<T>**

Here, T specifies the type of objects being compared

30. List any four overloaded forms/signatures of the binarySearch() method with their proper syntax.

static int binarySearch(byte *array*[ ], byte *value*)
static int binarySearch(char *array*[ ], char *value*)
static int binarySearch(double *array*[ ], double *value*)

static int binarySearch(float *array*[ ], float *value*)
static int binarySearch(int *array*[ ], int *value*)
static int binarySearch(long *array*[ ], long *value*)
static int binarySearch(short *array*[ ], short *value*)
static int binarySearch(Object *array*[ ], Object *value*)
static <T> int binarySearch(T[ ] *array*, T *value*, Comparator<? super T> *c*)

Here, *array* is the array to be searched, and *value* is the value to be located.

31. Differentiate Vector and Arrays

- **Vector** implements a dynamic array. It is similar to ArrayList, but with two differences:
- Vector is synchronized, and it contains many legacy methods that are not part of the Collections Framework.
- With the advent of collections, Vector was reengineered to extend AbstractList and to implement the List interface. Vector is fully compatible with collections, and a Vector can have its contents iterated by the enhanced for loop. Vector is declared like this: **class Vector<E>** Here, E specifies the type of element that will be stored. Here are the Vector constructors:
- The **Arrays** class provides various methods that are useful when working with arrays.
- These methods help bridge the gap between collections and arrays.
- The **asList( )** method returns a List that is backed by a specified array. In other words, both the list and the array refer to the same location.

32. What is the relationship between the Dictionary class and the Map interface in Java?

- **Dictionary** is an abstract class that represents a key/value storage repository and operates much like Map.
- Given a key and value, you can store the value in a Dictionary object. Once the value is stored, you can retrieve it by using its key.
- Dictionary was made generic. It is declared as shown here: **class Dictionary<K, V>**

Here, K specifies the type of keys, and V specifies the type of values.

- To add a key and a value, use the put( ) method.
- Use get( ) to retrieve the value of a given key.
- The keys and values can each be returned as an Enumeration by the keys( ) and elements( ) methods, respectively.

33. What are two advantages of using the MVC architecture in Java applications?

- **Separation of Concerns:** MVC promotes a clear separation of concerns between the model, view, and controller components. This separation allows for better code organization, improved modularity, and easier maintenance. Developers can focus on specific aspects of the application without impacting other components.
- **Code Reusability:** By separating the concerns into distinct components, code reuse becomes more feasible. The model can be reused across different views, and multiple views can be created for a single model. This reusability reduces duplication of code and improves development efficiency.
- **Testability:** The separation of concerns in MVC makes unit testing and debugging more manageable. Each component can be independently tested, as they have well-defined responsibilities and interfaces. This promotes comprehensive testing, reduces dependencies, and improves the overall quality of the application.
- **Enhanced User Experience:** With MVC, the view layer handles the presentation of data to the user. This separation allows for greater control over the user interface and enables the use of different views for different platforms or devices. Developers can create responsive and user-friendly interfaces tailored to specific user needs.

34. What is Model-View-Controller?

- The Model-View-Controller (MVC) architecture in Java is a design pattern that provides a structured approach for developing applications.

- It separates the application's concerns into three main components: the model, the view, and the controller.
- Each component has a specific role and responsibility within the architecture

35. What are two responsibilities of the View component in MVC?

View: The view is responsible for rendering the user interface and displaying the data to the user.

- It presents the data from the model to the user in a visually appealing and understandable way.
- The view component does not contain any business logic but instead relies on the model for data.

36. What are the primary roles of the Controller component in the MVC architecture?

Controller: The controller acts as an intermediary between the model and the view.

- It handles user input, processes user actions, and updates the model or view accordingly.
- The controller interprets user actions and triggers the appropriate methods in the model or view.
- It ensures the separation of concerns by keeping the view and model independent of each other.

37. What are the responsibilities of the Model component in handling data and business logic?

Model: The model represents the data and business logic of the application.

- It encapsulates the application's data and provides methods for accessing, manipulating, and updating that data.
- The model component is independent of the user interface and focuses solely on the application's functionality.

(4 to 6 marks)

1. Explain the benefits of Generics in Collections Framework?

- All collections are now generic, and many of the methods that operate on collections take generic type parameters.
- **Generics add the one feature that collections had been missing: type safety.** Prior to generics, all collections stored **Object** references, which meant that any collection could store any type of object.
- Thus, it was possible to accidentally store incompatible types in a collection.
- Doing so could result in run-time type mismatch errors.
- With generics, it is possible to explicitly state the type of data being stored, and run-time type mismatch errors can be avoided.
- **One other point: to gain the advantages that generics bring collections, older code will need to be rewritten.**
- This is also important because pre-generics code will generate warning messages when compiled by a modern Java compiler.
- To eliminate these warnings, you will need to add type information to all your collections code.
- The addition of generics fundamentally improves the usability and safety of collections because it
- Ensures that only references to objects of the proper type can actually be stored in a collection. Thus, a collection will always contain references of a known type.
- Eliminates the need to cast a reference retrieved from a collection. Instead, a reference retrieved from a collection is automatically cast into the proper type. This prevents run-time errors due to invalid casts and avoids an entire category of errors

2. List any five methods of Collection interface with its purpose.

- **Collection** is a generic interface that has this declaration: **interface Collection<E>**

E specifies the type of objects that the collection will hold.

- Collection extends the **Iterable** interface.
- Objects are added to a collection by calling **add( ).**

- You can add the entire contents of one collection to another by calling **addAll( )**.
- You can remove an object by using **remove( )**
- To remove a group of objects, call **removeAll( )**
- You can remove all elements except those of a specified group by calling **retainAll( )**.
- To empty a collection, call **clear( )**
- You can determine whether a collection contains a specific object by calling **contains( )**.
- To determine whether one collection contains all the members of another, call **containsAll( )**.
- You can determine when a collection is empty by calling **isEmpty( )**.
- The number of elements currently held in a collection can be determined by calling **size( )**
- Two collections can be compared for equality by calling **equals( )**.
- The precise meaning of "equality" may differ from collection to collection.
- For example, you can implement **equals( )** so that it compares the values of elements stored in the collection
- The **toArray( )** methods return an array that contains the elements stored in the invoking collection.
- The **Object[ ] toArray( )** returns an array of **Object**
- **<T> T[ ] toArray(T array[ ])** returns an array of elements that have the same type as the array specified as a parameter.

3. What are the basic interfaces of Java Collections Framework? Discuss the appropriate use of any four interfaces

- The Collections Framework defines several interfaces.
- Beginning with the collection interfaces is necessary because they determine the fundamental nature of the collection classes.

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends Queue to handle a double-ended queue.

	(Added by Java SE 6.)
List	Extends Collection to handle sequences (lists of objects).
NavigableSet	Extends SortedSet to handle retrieval of elements based on closest-match searches. (Added by Java SE 6.)
Queue	Extends Collection to handle special types of lists in which elements are removed only from the head.
Set	Extends Collection to handle sets, which must contain unique elements.
SortedSet	Extends Set to handle sorted sets.

4. List any five methods of List interface with its purpose.

Method	Description
void add(int index, E obj)	Inserts obj into the invoking list at the index passed in index. Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
boolean addAll(int index, Collection<? extends E> c)	Inserts all elements of c into the invoking list at the index passed in index. Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns true if the invoking list changes and returns false otherwise.
E get(int index)	Returns the object stored at the specified index within the invoking collection.
int indexOf(Object obj)	Returns the index of the first instance of obj in the invoking list. If obj is not an element of the list, –1 is returned.
int lastIndexOf(Object obj)	Returns the index of the last

	instance of obj in the invoking list. If obj is not an element of the list, -1 is returned.
ListIterator<E> listIterator( )	Returns an iterator to the start of the invoking list.
ListIterator<E> listIterator(int index)	Returns an iterator to the invoking list that begins at the specified index.
E remove(int index)	Removes the element at position index from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
E set(int index, E obj)	Assigns obj to the location specified by index within the invoking list.
List<E> subList(int start, int end)	Returns a list that includes elements from start to end-1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

5.Explain any four methods defined by NavigableSet interface

Method	Description
E ceiling(E obj)	Searches the set for the smallest element e such that e >= obj. If such an element is found, it is returned. Otherwise, null is returned.
Iterator<E> descendingIterator( )	Returns an iterator that moves from the greatest to least. In other words, it returns a reverse iterator.
NavigableSet<E> descendingSet( )	Returns a NavigableSet that is the reverse of the invoking set. The resulting set is backed by the invoking set.

E floor(E obj)	Searches the set for the largest element e such that e <= obj. If such an element is found, it is returned. Otherwise, null is returned.
NavigableSet<E> headSet(E upperBound, boolean incl)	Returns a NavigableSet that includes all elements from the invoking set that are less than upperBound. If incl is true, then an element equal to upperBound is included. The resulting set is backed by the invoking set.
E higher(E obj)	Searches the set for the largest element e such that e > obj. If such an element is found, it is returned. Otherwise, null is returned.

6. List any five methods of Queue interface with its purpose.

Method	Description
E element( )	Returns the element at the head of the queue. The element is not removed. It throws **NoSuchElementException** if the queue is empty.
boolean offer(E obj)	Attempts to add obj to the queue. Returns true if obj was added and false otherwise.
E peek( )	Returns the element at the head of the queue. It returns null if the queue is empty. The element is not removed.
E poll( )	Returns the element at the head of the queue, removing the element in the process. It returns null if the queue is empty.
E remove( )	Removes the element at the head of the queue, returning the element in the process. It throws NoSuchElementException if the queue is empty.

7. List any five methods of Deque interface with its purpose.

Method	Description
void addFirst(E obj)	Adds obj to the head of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
void addLast(E obj)	Adds obj to the tail of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
E getFirst( )	Returns the first element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
E getLast( )	Returns the last element in the deque. The object is not removed from the deque. It throws NoSuchElementException if the deque is empty.
E peekFirst( )	Returns the element at the head of the deque. It returns null if the deque is empty. The object is not removed.
E peekLast( )	Returns the element at the tail of the deque. It returns null if the deque is empty. The object is not removed.
E pollFirst( )	Returns the element at the head of the deque, removing the element in the process. It returns null if the deque is empty.
E pollLast( )	Returns the element at the tail of the deque, removing the element in the process. It returns null if the deque is empty.
E pop( )	Returns the element at the head of

	the deque, removing it in the process. It throws NoSuchElementException if the deque is empty.
void push(E obj)	Adds obj to the head of the deque. Throws an IllegalStateException if a capacity-restricted deque is out of space.
E removeFirst( )	Returns the element at the head of the deque, removing the element in the process. It throws NoSuchElementException if the deque is empty.
E removeLast( )	Returns the element at the tail of the deque, removing the element in the process. It throws NoSuchElementException if the deque is empty.

8. With an example explain the usage of ArrayList

- Java ArrayList class uses a *dynamic array* for storing the elements.
- It is like an array, but there is *no size limit.* We can add or remove elements anytime. So, it is much more flexible than the traditional array.
- The ArrayList in Java can have the duplicate elements also.
- It implements the List interface so we can use all the methods of List interface here.
- **ArrayList<String> list=new ArrayList<String>();**

```
//This program demonstrates the ArrayList
import java.util.*;
public class ArrayListExample1{
public static void main(String args[]){ ArrayList<String>
list=new ArrayList<String>();
list.add("Mango");//Adding object in arraylist
list.add("Apple");
list.add("Banana");
list.add("Grapes");
//Printing the arraylist object
```

```
System.out.println(list);
}
}output: [Mango, Apple, Banana, Grapes]
```

9. Write a Java program that demonstrates how to convert an ArrayList to an array using the toArray() method

```
// Convert an ArrayList into an array.
import java.util.*;
class ArrayListToArray {
public static void main(String args[]) {
// Create an array list.
ArrayList al = new ArrayList();
// Add elements to the array list.
al.add(1);        al.add(2);        al.add(3);        al.add(4);
System.out.println("Contents of al: " + al);
// Get the array.
Integer ia[] = new Integer[al.size()];
ia = al.toArray(ia);
int sum = 0;
// Sum the array.
for(int i : ia)
sum += i;
System.out.println("Sum is: " + sum);
}
} The output : Contents of al: [1, 2, 3, 4]   Sum is: 10
```

10. Write a Java program that demonstrates the usage of the LinkedList class from the Java Collections Framework.

```
// Demonstrate LinkedList.
import java.util.*;
class LinkedListDemo {
public static void main(String args[]) {
// Create a linked list.
LinkedList ll = new LinkedList();
// Add elements to the linked list.
ll.add("F"); ll.add("B"); ll.add("D"); ll.add("E"); ll.add("C");
ll.addLast("Z"); ll.addFirst("A");
```

```
        ll.add(1, "A2");
        System.out.println("Original contents of ll: " + ll);
        // Remove elements from the linked list. ll.remove("F");
        ll.remove(2);
    System.out.println("Contents of ll after deletion: " + ll);
    // Remove first and last elements.
    ll.removeFirst(); ll.removeLast();
    System.out.println("ll after deleting first and last: " + ll);
    // Get and set a value
    String val = ll.get(2);
    ll.set(2, val + " Changed");
    System.out.println("ll after change: " + ll); }
    ; output:
    Original contents of ll: [A, A2, F, B, D, E, C, Z]
    Contents of ll after deletion: [A, A2, D, E, C, Z]
    ll after deleting first and last: [A2, D, E, C]
    ll after change: [A2, D, E Changed, C]
```

11. Explain four constructors of the HashSet class from the Java Collections Framework, including their parameters

- The advantage of hashing is that it allows the execution time of add( ), contains( ), remove( ), and size( ) to remain constant even for large sets
- The following constructors are defined:

  HashSet( )
  HashSet(Collection<? extends E> c)
  HashSet(int capacity)
  HashSet(int capacity, float fillRatio)

- The first form constructs a default hash set.
- The second form initializes the hash set by using the elements of c.
- The third form initializes the capacity of the hash set to capacity. (The default capacity is 16.)
- The fourth form initializes both the capacity and the fill ratio (also called load capacity) of the hash set from its arguments.
- The fill ratio must be between 0.0 and 1.0, and it determines how full the hash set can be before it is resized upward.
- // Demonstrate HashSet.

```
import java.util.*;
class HashSetDemo {
public static void main(String args[]) {
// Create a hash set.
HashSet hs = new HashSet();
// Add elements to the hash set.
hs.add("B");
hs.add("A");
hs.add("D");
hs.add("E");
hs.add("C");
hs.add("F");
System.out.println(hs); } }
```

output from this program:

[D, A, F, C, B, E]

12. What is an iterator? Explain with an example.

- Each of the collection classes provides an iterator( ) method that returns an iterator to the start of the collection.
- By using this iterator object, you can access each element in the collection, one element at a time.
- In general, to use an iterator to cycle through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's iterator( ) method.
2. Set up a loop that makes a call to hasNext( ). Have the loop iterate as long as hasNext( ) returns true.
3. Within the loop, obtain each element by calling next( ).

```
// Demonstrate iterators.
import java.util.*;
class IteratorDemo {
public static void main(String args[]) {
// Create an array list.
ArrayList<String> al = new ArrayList<String>();
// Add elements to the array list.
al.add("C");        al.add("A");        al.add("E");
al.add("B");        al.add("D");        al.add("F");
// Use iterator to display contents of al.
```

```
System.out.print("Original contents of al: ");
Iterator<String> itr = al.iterator();
while(itr.hasNext()) {
String element = itr.next();
System.out.print(element + " ");
}
System.out.println();
// Modify objects being iterated.
ListIterator<String> litr = al.listIterator();
while(litr.hasNext()) {
String element = litr.next();
litr.set(element + "+");
}
System.out.print("Modified contents of al: ");
itr = al.iterator();
while(itr.hasNext()) {
String element = itr.next();
System.out.print(element + " ");        }
System.out.println();
// Now, display the list backwards.
System.out.print("Modified list backwards: ");
while(litr.hasPrevious()) {
String element = litr.previous();
System.out.print(element + " ");
}
System.out.println();}      }
```

The output is shown here:

Original contents of al: C A E B D F

Modified contents of al: C+ A+ E+ B+ D+ F+

Modified list backwards: F+ D+ B+ E+ A+ C+

13. Explain the usage of the for-each loop in Java when working with collections. Compare and contrast the for-each loop with the traditional approach of using an Iterator.

- If you won't be modifying the contents of a collection or obtaining elements in reverse order, then the for-each version of the for loop is often a more convenient

alternative to cycling through a collection than is using an iterator.

- Because all of the collection classes implement this interface, they can all be operated upon by the for.
- The following example uses a for loop to sum the contents of a collection:

```
// Use the for-each for loop to cycle through a collection.
import java.util.*;
class ForEachDemo {
public static void main(String args[]) {
// Create an array list for integers.
ArrayList<Integer> vals = new ArrayList<Integer>();
// Add values to the array list.
vals.add(1);    vals.add(2);        vals.add(3);
vals.add(4);      vals.add(5);
// Use for loop to display the values.
System.out.print("Original contents of vals: ");
for(int v : vals)
System.out.print(v + " ");
System.out.println();
// Now, sum the values by using a for loop.
int sum = 0;
 for(int v : vals)
sum += v;
System.out.println("Sum of values: " + sum);
}
 }
```

The output from the program is shown here:

```
Original contents of vals: 1 2 3 4 5
Sum of values: 15
```

14. Explain how to store objects of user-defined classes in Java collections like ArrayList

Storing User-Defined Classes in Collections

- For the sake of simplicity, the foregoing examples have stored built-in objects, such as String or Integer, in a collection.
- Of course, collections are not limited to the storage of built-in objects.

- The power of collections is that they can store any type of object, including objects of classes that you create.
- For example, consider the following example that uses a LinkedList to store mailing addresses:
- **// A simple mailing list example.**

```
import java.util.*;
class Address {
private String name; private String street; private String city; private String state; private String code;
Address(String n, String s, String c, String st, String cd) {
name = n; street = s; city = c;  state = st; code = cd; }
public String toString() {
return name + "\n" + street + "\n" + city + " " + state + " " + code; }  }
class MailList {
public static void main(String args[]) {
LinkedList ml = new LinkedList();
// Add elements to the linked list.
ml.add(new Address("J.W. West", "11 Oak Ave", "Urbana", "IL", "61801"));
ml.add(new Address("Ralph Baker", "1142 Maple Lane", "Mahomet", "IL", "61853")); ml.add(new Address("Tom Carlton", "867 Elm St", "Champaign", "IL", "61820"));
// Display the mailing list. for(Address element : ml) System.out.println(element + "\n"); System.out.println(); } }
```

The output from the program is shown here: J.W. West

```
11 Oak Ave
Urbana IL 61801
Ralph Baker
1142 Maple Lane
Mahomet IL 61853
Tom Carlton
867 Elm St
Champaign IL 61820
```

15. Write how Vector is differ from ArrayList.

- **Vector** implements a dynamic array. It is similar to ArrayList, but with two differences:
- Vector is synchronized, and it contains many legacy methods that are not part of the Collections Framework.

- With the advent of collections, Vector was reengineered to extend **AbstractList** and to implement the **List** interface. it was retrofitted for generics and reengineered to implement **Iterable**. This means that Vector is fully compatible with collections, and a Vector can have its contents iterated by the enhanced for loop.
- Vector is declared like this: **class Vector<E>**
  Here, E specifies the type of element that will be stored.
  Here are the Vector constructors:
  Vector( ) creates a default vector which has an initial size of 10
  Vector(int *size*) creates a vector whose initial capacity is specified by *size*
  Vector(int *size*, int *incr*) creates a vector whose initial capacity is specified by *size* and whose increment is specified by *incr*. The increment specifies the number of elements to allocate each time that a vector is resized upward.
  Vector(Collection<? extends E> *c*) creates a vector that contains the elements of collection *c*.
  Because **Vector** Implements **List**, you can use a vector just like you use an **ArrayList** instance. You can also manipulate one using its legacy methods

16. Write the usage of any four methods of Map interface

Four methods of Map interfaces are:

void clear( )	Removes all key/value pairs from the invoking map.
Set<Map.Entry<K, V>> entrySet( )	Returns a Set that contains the entries in the map. The set contains objects of type Map.Entry. Thus, this method provides a set-view of the invoking map.
V get(Object k)	Returns the value associated with the key k. Returns null if the key is not found
boolean equals(Object obj)	Returns true if obj is a Map and contains the same entries. Otherwise, returns false.
boolean isEmpty( )	Returns true if the invoking map is empty. Otherwise, returns false.

V put(K k, V v)	Puts an entry in the invoking map, overwriting any previous value associated with the key. The key and value are k and v, respectively. Returns null if the key did not already exist. Otherwise, the previous value linked to the key is returned
int size( )	Returns the number of key/value pairs in the map

17. What is a map?

- A map is an object that stores associations between keys and values, or key/value pairs. Given a key, you can find its value.
- Both keys and values are objects.
- The keys must be unique, but the values may be duplicated.
- Some maps can accept a null key and null values, others cannot.
- Map is generic and is declared as shown here:
- **interface Map<K, V>**
- Here, K specifies the type of keys, and V specifies the type of values.

18. Write a Java program that demonstrates the usage of the HashMap class from the Java Collections Framework

```
import java.util.*;
class HashMapDemo {
public static void main(String args[]) {
// Create a hash map.
HashMap<String, Double> hm = new HashMap<String,
Double>();
// Put elements to the map
hm.put("John Doe", new Double(3434.34));
hm.put("Tom Smith", new Double(123.22));
hm.put("Jane Baker", new Double(1378.00));
hm.put("Tod Hall", new Double(99.22));
hm.put("Ralph Smith", new Double(-19.08));
// Get a set of the entries.
```

```
Set<Map.Entry<String, Double>> set = hm.entrySet();
// Display the set.
for(Map.Entry<String, Double> me : set) {
System.out.print(me.getKey() + ": ");
System.out.println(me.getValue());          }
System.out.println();
// Deposit 1000 into John Doe's account.
double balance = hm.get("John Doe");
hm.put("John Doe", balance + 1000);
System.out.println("John Doe's new balance: " +
hm.get("John Doe"));}  }
```

Output from this program is shown here (the precise order may vary):

```
Ralph Smith: -19.08
Tom Smith: 123.22
John Doe: 3434.34
Tod Hall: 99.22
Jane Baker: 1378.0
John Doe's new balance: 4434.34
```

19. Write a Java program that demonstrates the usage of a custom Comparator for sorting strings in reverse order.

```
// Use a custom comparator.
import java.util.*;
// A reverse comparator for strings.
class MyComp implements Comparator<String> {
public int compare(String a, String b) {
String aStr, bStr;
aStr = a;
bStr = b;
// Reverse the comparison.
return bStr.compareTo(aStr);
}  // No need to override equals.
}
```

20.

```
class CompDemo {
public static void main(String args[]) {
// Create a tree set.
TreeSet<String> ts = new TreeSet<String>(new
```

```
MyComp());
// Add elements to the tree set.
ts.add("C");
ts.add("A");
ts.add("B");
ts.add("E");
ts.add("F");
ts.add("D");
// Display the elements.
for(String element : ts)
System.out.print(element + " ");
System.out.println();
}
```

As the following output shows, the tree is now stored in reverse order:F E D C B A

21. Write the usage of any 4 collection algorithms.

- The Collections Framework defines several algorithms that can be applied to collections and maps.
- These algorithms are defined as static methods within the Collections class.
- Methods include **checkedSet( )**, **checkedList( )**, **checkedMap( )**, and so on.
- They obtain a type-safe view for the indicated collection.
- **Collections** defines three static variables: **EMPTY_SET**, **EMPTY_LIST**, and **EMPTY_MAP**. All are immutable
- The set of methods that begins with **unmodifiable** returns views of the various collections that cannot be modified.
- These will be useful when you want to grant some process read—but not write—capabilities on a collection
- The **reverseOrder( )** method returns a **Comparator** that reverses the comparison of **Integer** objects.
- The list elements are sorted according to this comparator and then are displayed

22. Write a program to convert a given array into a collection with the asList() method.

The asList( ) method returns a List that is backed by a specified array. In other words,both the list and the array refer to the same location.

```
// Demonstrate Arrays
import java.util.*;
class ArraysDemo {
public static void main(String args[]) {
// Allocate and initialize array.
int array[] = new int[10];
for(int i = 0; i < 10; i++)
array[i] = -3 * i;
// Display, sort, and display the array.
System.out.print("Original contents: ");
display(array);
Arrays.sort(array);
System.out.print("Sorted: ");
display(array);
// Fill and display the array.
Arrays.fill(array, 2, 6, -1);
System.out.print("After fill(): ");
display(array);
// Sort and display the array.
Arrays.sort(array);
System.out.print("After sorting again: ");
display(array);
// Binary search for -9.
System.out.print("The value -9 is at location ");
int index =
Arrays.binarySearch(array, -9);
System.out.println(index);
23. }
static void display(int array[]) {
for(int i: array)
System.out.print(i + " ");
System.out.println();
}
}The following is the output from this program:
Original contents: 0 -3 -6 -9 -12 -15 -18 -21 -24 -27
Sorted: -27 -24 -21 -18 -15 -12 -9 -6 -3 0
After fill(): -27 -24 -1 -1 -1 -1 -9 -6 -3 0
```

After sorting again: -27 -24 -9 -6 -3 -1 -1 -1 -1 0
The value -9 is at location 2

24. Explain any four legacy methods of vector

- Vector implements List, you can use a vector just like you use an **ArrayList** instance. You can also manipulate one using its legacy methods. For example, after you instantiate a Vector, you can add an element to it by calling **addElement( ).**
- To obtain the element at a specific location, call **elementAt( ).**
- To obtain the first element in the vector, call **firstElement( ).**
- To retrieve the last element, call **lastElement( ).**
- You can obtain the index of an element by using **indexOf( )** and **lastIndexOf( ).**
- To remove an element, call **removeElement( )**or **removeElementAt()**

25. Explain the roles and responsibilities of the Model, View, and Controller components in the MVC architecture.

Model: The model represents the data and business logic of the application.

- It encapsulates the application's data and provides methods for accessing, manipulating, and updating that data.
- The model component is independent of the user interface and focuses solely on the application's functionality.

View: The view is responsible for rendering the user interface and displaying the data to the user.

- It presents the data from the model to the user in a visually appealing and understandable way.
- The view component does not contain any business logic but instead relies on the model for data.

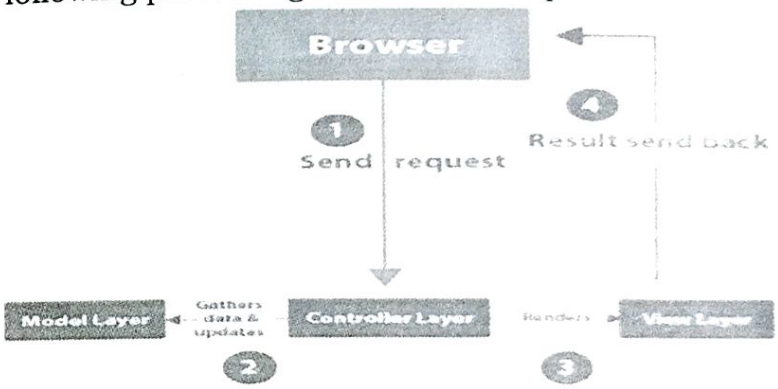
Controller: The controller acts as an intermediary between the model and the view.

- It handles user input, processes user actions, and updates the model or view accordingly.
- The controller interprets user actions and triggers the appropriate methods in the model or view.

- It ensures the separation of concerns by keeping the view and model independent of each other.

26. Explain the flow of execution when a user interacts with an MVC-based Java web application. Walk through the steps involved, starting from the user's request, the role of the controller, the interaction with the model and view, and finally, the response sent back to the user.

- In Java programming, the Model comprises basic Java classes that encapsulate data and business logic.
- The View is responsible for presenting the data to the user interface, while the Controller consists of servlets that handle user requests.
- This clear separation of components enables the following processing flow for user requests:



In the context of the server-client architecture, the process of handling a page request can be described as follows:

- A client, typically a web browser, initiates a request and sends it to the server-side controller.
- The controller receives the request and interacts with the model component. It retrieves the necessary data from the model, which may involve processing and manipulating the data as required.
- Once the controller has gathered the requested data, it transfers this data to the view layer.
- The view layer, utilizing the provided data, generates the appropriate output or representation of the requested page. Finally, the generated result is sent back to the client's browser, completing the request-response cycle.